

HW3# Cache Optimization

鄭維, 10801069

Abstract—This report analyzes and optimizes the data cache in the Aquila SoC. Key improvements include adjusting associativity and cache size, refining dirty bit management, upgrading replacement policies, improving FSM, and introducing a stride-based prefetching algorithm. Using a program to compute π to 5000 digits, performance was measured by hit rate, miss latency, and efficiency. The best configuration showed a 16.11% improvement over the baseline, completing in 24,395 ms. Eliminating write-back latency greatly enhanced performance, while other optimizations offered additional gains. The findings emphasize trade-offs in cache design and suggest further refinements to maintain efficiency under diverse access patterns.

I. INTRODUCTION

This report focuses on optimizing the data cache in the Aquila SoC, leveraging a systematic approach to identify and address key inefficiencies. The initial analysis targeted associativity and cache size configurations to balance performance and resource utilization. Subsequently, enhancements were implemented in dirty bit management, replacement policies, and the finite state machine (FSM) controlling the cache. A prefetching algorithm based on stride detection was also introduced to reduce cache miss. These changes were rigorously tested using a program to compute π , providing valuable insights into the effectiveness of each optimization.

II. METHODOLOGY

This section outlines the sequential steps taken to optimize the data cache behavior within the **aquila dram**.

A. Profiler

To analyze cache behavior, a custom module ‘dcache_profiler.v’ was integrated into the design. This module collected detailed statistics, including cache hit and miss rates, as well as average latency for different memory operations.

B. Cache Configuration Analysis

The analysis included exhaustive combinations of associative and cache size configurations. Specifically, the cache was tested with associative of 2, 4, and 8 ways and sizes of 4 KB and 8 KB.

C. Dirty Bit Management

The dirty bit mechanism was already implemented in **aquila_dram** but could be optimized: once set, the dirty bit remained uncleared until a system reset. This led to every cache miss triggering a write-back operation, incurring unnecessary performance penalties. Addressing this issue was a key focus.

D. Replacement Policy Enhancement

The default replacement policy of **aquila_dram** used FIFO, which was replaced with an LRU policy to improve efficiency. A 'lru_matrix.v' module was implemented to maintain the LRU information for each cache line using a matrix corresponding to the associative level.

E. FSM Improvement

The finite state machine (FSM) of the data cache controller was refined by removing the **WbToMemFinish** state. This change allowed smoother transitions from **WbToMem** to **RdFromMem** upon receiving a ready signal from the memory, thereby reducing unnecessary stalls.

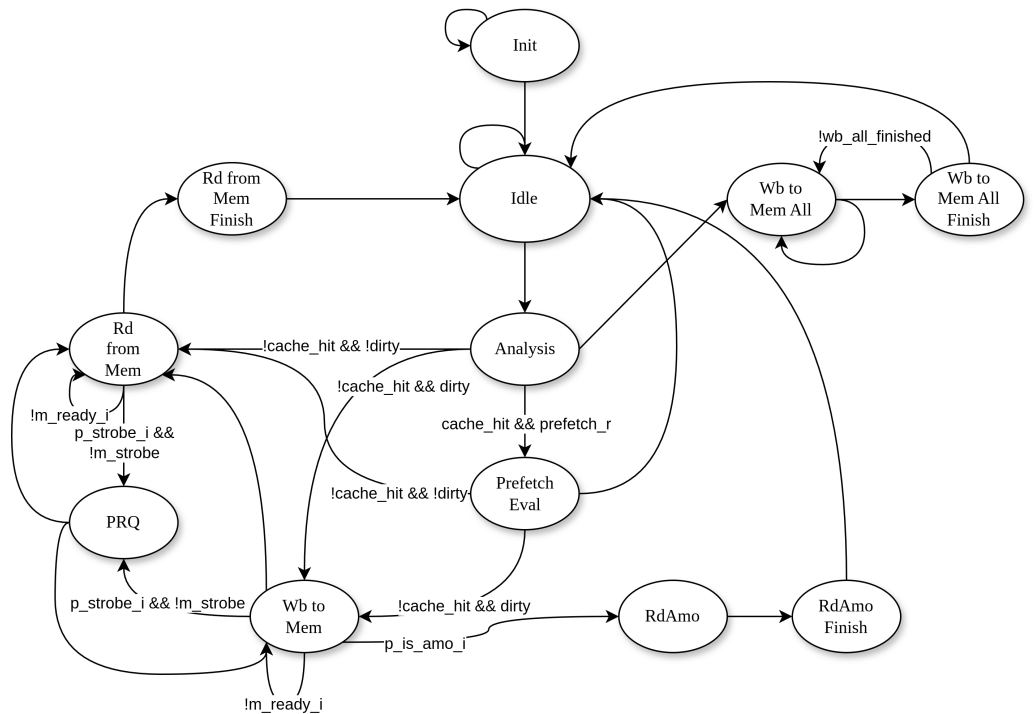


Fig. 1. Modified FSM

F. Prefetching Algorithm

To improve cache hit rates, prefetching was implemented based on stride detection. A stride is identified when consecutive line address differences are consistent. During

prefetching, memory remains busy while processor requests are handled by temporarily switching to the PRQ (Prefetching Request) state. Cache hits are handled immediately, while cache misses are paused until prefetching completes, allowing the system to reanalyze as the prefetched line might match the request. The adjusted FSM is shown in Fig. 1.

G. Experimental Setup

The performance was evaluated by measuring the time taken to compute π to 5000 digits using the program 'pi.c', compiled with GCC 13.2.0. Each experiment was executed six times, with one initial warm-up run excluded from the calculations. The results presented are the average values of the remaining five executions.

III. RESULTS

The improvement is computed according to the following formula:

$$\frac{T-T'}{T} \quad (1)$$

Here, T represents either performance (ms) or latency (cycles per operation), with a prime symbol (') indicating the improved value.

TABLE I. CACHE CONFIGURATION ANALYSIS

Cache Size/ Associative	4 KB		
	Performance (ms)	Hit Rate (%)	Miss Latency (cycles per operation)
2-way	30534	79.97	54.65
4-way	30517	79.97	54.58
8-way	30517	79.97	54.58
Cache Size	8 KB		
2-way	25445	87.42	54.58
4-way	29079	82.08	54.58
8-way	29860	80.92	54.55

TABLE II. DIRTY BIT OPTIMIZED

Cache Size/ Associative	4 KB		
	Performance (ms)	Hit Rate (%)	Miss Latency (cycles per operation)
2-way	27968	79.97	44.38
4-way	27957	79.97	44.34
8-way	27972	79.97	44.40
Cache Size	8 KB		
2-way	24395	87.42	47.89
4-way	26913	82.08	44.89
8-way	27434	80.92	44.35

TABLE III. MISS LATENCY

Cache Size/ Adjustment	4 KB	
	Miss Latency (cycles per operation)	Improvement ^a (%)
Original	54.58	0.00
Dirty Bit Optimized	44.34	18.76
FSM Improved	43.73	19.87
Cache Size	8 KB	
Original	54.58	0.00
Dirty Bit Optimized	44.89	17.75
FSM Improved	44.19	19.03

^a. Comparison to original (FIFO 4-way associative, 4 KB or 8 KB)

TABLE IV. HIT RATE

Configuration	8 KB 4-way Associative		
	FIFO	LRU	Prefetch
Hit Rate (%)	82.08	82.18	82.48
Read Hit Rate (%)	80.58	80.58	79.21
Write Hit Rate (%)	93.36	84.17	83.30

TABLE V. BEST IMPROVEMENT

Configuration	8 KB 2-way Associative		
	FIFO	LRU	Prefetch
Performance (ms)	24395	24863	24867
Hit Rate (%)	87.42	85.92	85.93
Write Hit Rate (%)	93.36	89.77	89.77
Improvement ^b (%)	16.11	14.50	14.48

^b. Comparison to original (FIFO 4-way associative, 4 KB or 8 KB)

IV. DISCUSSION

A. Cache Configuration Analysis

There's no significant change in adjusting associativity while fixing the cache size at 4 KB. However, when fixing the cache size at 8 KB, the 2-way associative configuration has the best performance, followed by 4-way, and finally 8-way. This could be attributed to the improvement in hit rate across various associativity configurations. The significant change observed in 8 KB cache size is likely due to having more cache lines, which better reflect changes in the number of lines. Since smaller associativity leads to more cache lines, the impact is more pronounced.

B. Miss Latency

Modified dirty bit management reduced the miss latency per operation across all configurations, drastically improving overall performance. FSM improvements also contributed to the reduction of miss latency.

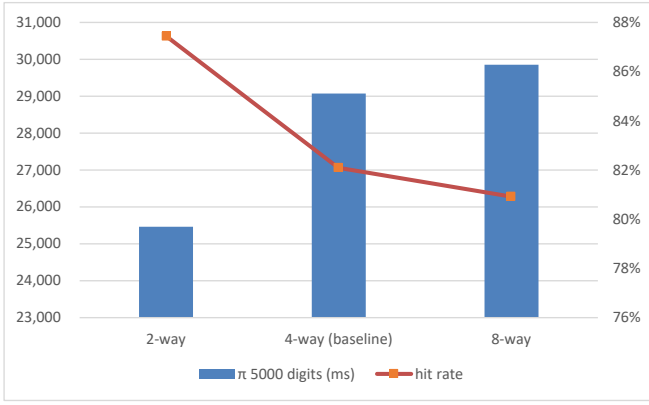


Fig. 2. 8 KB FIFO

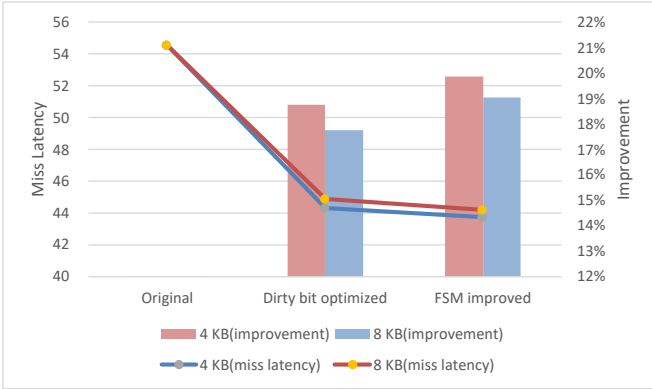


Fig. 3. 4-way Associative

C. Hit Rate

LRU and prefetching improved the hit rate in the 8 KB cache size configuration. However, there was no significant change in the 4 KB cache size configuration.

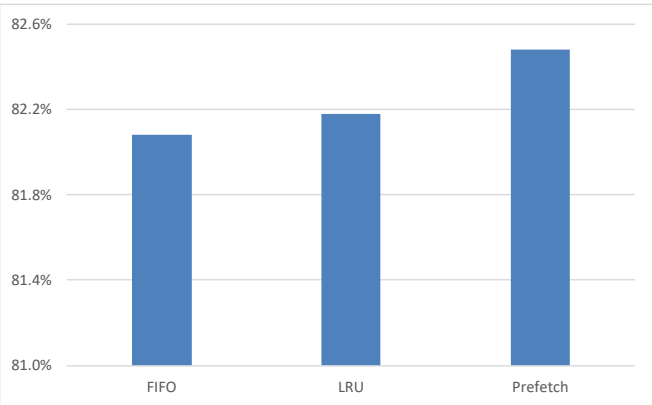


Fig. 4. 4-way Associative

D. Best Improvement

From the above results, the hit rate improved through the transition from FIFO to LRU, the prefetch algorithm, and the shift from 4-way to 2-way associativity (specifically in the 8 KB cache). Miss latency improvements were achieved through optimized dirty bit management and FSM enhancements. Assuming the best configuration is derived from the above combinations, it is noted that FIFO to LRU and the prefetch algorithm did not improve the hit rate in the 8 KB 2-way associative configuration. This may be because the 8 KB 2-way associative configuration already had a high hit rate due to its superior write hit rate. As a result, FIFO to LRU and prefetching could not further enhance the write hit rate and instead decreased it. The best improvement observed was with the 8 KB, 2-way associative configuration using FIFO and optimized dirty bit management, compared to the baseline of 8 KB, 4-way associative configuration using FIFO.

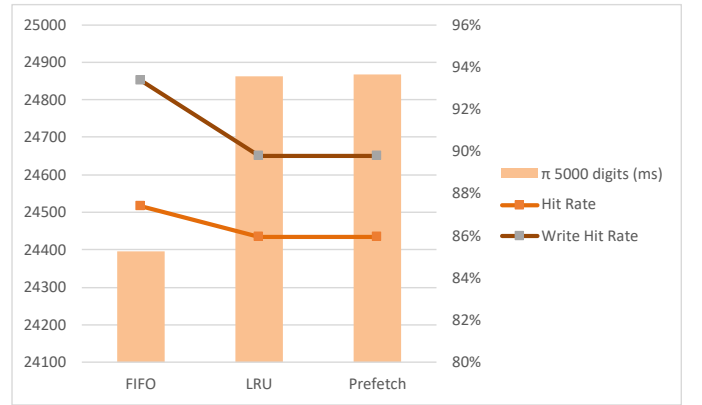


Fig. 5. 8 KB 2-way Associative

V. CONCLUSION

Memory access patterns are complex, making the prefetching mechanism prone to inefficiency in some scenarios, while eliminating write-back latency significantly boosts performance. Deciding between these approaches requires careful consideration of trade-offs due to limited resources.

For further improvement, we could consider to add a write buffer and altering the FSM order (executing **RdFromMem** before **WbToMem**) to allow write-back operations to be performed in the background without stalling the processor. Besides, introducing a prefetch flushing feature to prioritize processor requests over prefetching to reduce hit latency. However, since the hit latency penalty during prefetching is minor, the focus should remain on enhancing the accuracy of the prefetching logic to improve the hit rate further.